How accurately can a neural network identify different Minecraft items?

Page Count: 18

May 2023

Introduction

A neural network is a multi-layered network consisting of input, output, and hidden layers (IBM). The input layer contains the raw data that is inputted into it, while the output layer contains the processed data after going through the hidden layers. When sending data through hidden layers, a weight is applied to that data, which is then plugged into an activation function and sent to the output.

In this paper, a neural network is used for image recognition in order to identify the different types of items from Minecraft. When building a neural network to identify images, a set of training images is sent through the network for it to learn the patterns found in each item. The number of items that the image could possibly contain is the number of nodes within the output layer, therefore once the data is sent through the hidden layers, each output node holds a value between 0 and 1 representing the percentage of certainty that the network believes the item is in the image. Using this information, testing images can then be inputted into the network in which the network will decide the item that it thinks the image contains.

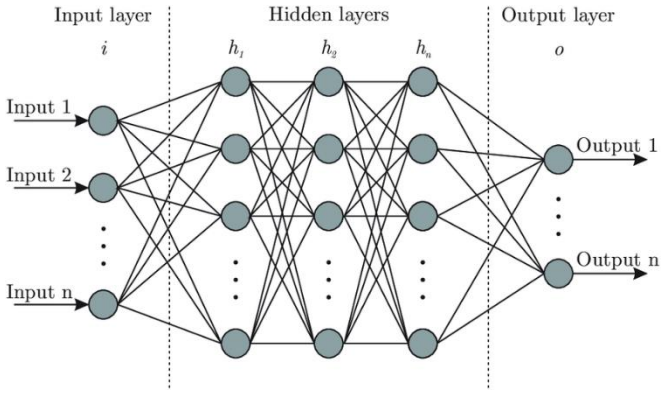


Figure 1. Example of layers in a neural network (Gavrilova).

Motivation

As someone interested in graphic design for games, the textures used within games are very important to the aesthetic and functionality of the game. Not only do the textures have a huge impact on whether a game looks good or not, but can also affect how easy it is for someone to play the game. For example, the idea that swords within games are used in fighting is very intuitive, but if a game decides to give the sword a completely different purpose, then it could easily evoke confusion among the players. The same concept is applied when altering the textures of a game like Minecraft, which is why this neural network can be very useful for those creating their own texture pack. When creating textures, it is important that the player can easily identify the item, therefore using this program to test whether that item looks similar enough to other texture packs can help determine whether that item can be easily recognized. I have created a few texture packs before, and this is a major consideration when determining if a texture is suitable for each item.

Collecting Data

To begin with, the images need to be converted into a black and white image, and then into a matrix. Doing this, this allows each value in the matrix to represent each pixel of the image on a scale of 0-255, with 0 being black and 255 being white. Converting the image into numbers allows it to be manipulated so that the neural network can make predictions on the item type.



```
[[[  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
  [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
  [  0   0   0   0   0   0   0  42  42  42  42  42   0   0   0   0]
  [  0   0   0   0   0   0  42 105  90  82  82  90  42  56  82   0]
  [  0   0   0   0   0   0   0  42  25  25  25  82  82 105  31   0]
  [  0   0   0   0   0   0   0   0   0   0   0  56  90  82  25   0]
  [  0   0   0   0   0   0   0   0   0   0   0  56  82  31  82  90]
  [  0   0   0   0   0   0   0   0   0   0  56 105  31   0  25  82]
  [  0   0   0   0   0   0   0   0  56  82  31   0   0  25  82  25]
  [  0   0   0   0   0   0   0  56 105  31   0   0   0  25  90  25]
  [  0   0   0   0   0   0  56  82  31   0   0   0   0  25 105  25]
  [  0   0   0   0   0  56 105  31   0   0   0   0   0   0  25   0]
  [  0   0   0   0  56  82  31   0   0   0   0   0   0   0   0   0]
  [  0   0   0  56 105  31   0   0   0   0   0   0   0   0   0   0]
  [  0   0   0  31  31   0   0   0   0   0   0   0   0   0   0   0]
  [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]]]
```

Figure 2. The image inputted compared to how the program interprets it.

This matrix is then flattened for all values to be contained within a one-dimensional array. This is done by appending all of arrays together in order of top to bottom and creating one long array.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} = [1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6]$$

The list of arrays that now represent the images are appended into one matrix where each row is an image. This matrix is then plugged into a transpose function in which the rows become columns (Math Insight).

$$[1 \quad 2 \quad 3]^T = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

All values are normalized by dividing each number by 255 for the range to be within $0 \leq x \leq 1$. This is done in order to decrease the complexity, which in result will increase the speed of computation needed (Zhang).

Since the textures for Minecraft items are 16 pixels in height and 16 pixels in length, the length of the input layer is 256, representing every pixel in the texture. Within this network, the hidden layer will contain 5 nodes, and will be testing for 5 different items. Each item will be indexed as a specific number in which the network can output a prediction as.

```
if 'shovel' in imgname:
    itemtype = 0
if 'axe' in imgname:
    itemtype = 1
if 'pickaxe' in imgname:
    itemtype = 2
if 'sword' in imgname:
    itemtype = 3
if 'hoe' in imgname:
    itemtype = 4
```

Figure 3. Code for indexing each item with a specific value.

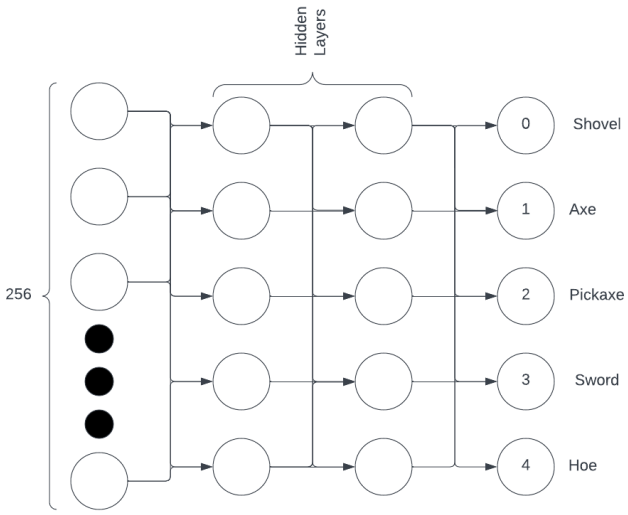


Figure 4. Items represented as the output values in the network.

Forward Propagation

Forward propagation is the process of calculating and storing values of each layer in the network (D2L). The values within each layer are manipulated in pursuit of achieving an accurate output layer. To begin with, a weight and bias value are needed. For the first iteration, both the weight and the bias contain random numbers between 0 and 1 (Divakar). The succeeding iterations will use weights and bias produced by the network. The purpose of the weight values is to multiply a value that is correspondent to the connection between each input node (256) and hidden layer node (5), therefore it results in a 256 by 5 matrix (Zhang). The bias acts as a constant that is corresponding with the hidden layer nodes and is added to the product of the weighted values, therefore adding a 5 by 1 matrix. This will result in the first hidden array ($z^{(1)}$). Let w = weight, a = layer matrix, b = bias, L = layer

$$z^{(L)} = w^{(L)} a^{(L-1)} + b^{(L)}$$

$$z^{(L)} = \begin{bmatrix} a_1 & \cdots & a_{256} \\ \vdots & \ddots & \vdots \\ a_5 & \cdots & a_{1280} \end{bmatrix} \cdot \begin{bmatrix} w_1 & \cdots & w_{256} \\ \vdots & \ddots & \vdots \\ w_5 & \cdots & w_{1280} \end{bmatrix} + \begin{bmatrix} b_1 \\ \vdots \\ b_5 \end{bmatrix}$$

The next step is the introduction of an activation function. The activation function is responsible for deciding whether a node is activated or not (Tiwari). The activation function is an important aspect of the layer as it introduces a nonlinear function to all the nodes in the layer, therefore allowing a more complex learning ability for the network. There are a variety of activations functions that can be used within neural networks that all have their different uses and levels of complexity to neural networks (Gupta). The complexity of the network majorly affects the network's learnability as well as the ability to generalize the learned information on new data. Many factors go into the complexity of a network such as the number of hidden layers, activation functions used, type of learning data used, and how much learning data is given. The activation function that is used for this neural network is the rectified linear unit (ReLU) function. With this function, the nodes are only activated if they are positive. ReLU as an activation function is useful as it doesn't activate all the nodes at once, therefore is more computationally efficient than other functions (Gupta). Let the ReLU function be represented by R.

$$R(z) = \begin{cases} 0, & z < 0 \\ z, & z \geq 0 \end{cases}$$

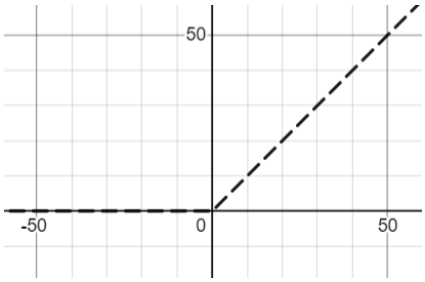


Figure 5. Graph of the ReLU function.

Afterwards, the output is inputted into the next hidden layer, where a weight and bias value is again added, this time in correlation to the connections between the hidden and the output layer.

For calculating the output layer, another activation function is added, which for this case the SoftMax activation function will be used. The SoftMax function is used in this instance since it's in the layer prior to the output layer, therefore needs to output a probability value (Gupta). Along with that, since it is also non-linear, it contributes to creating a more complex network.

$$\frac{e^{z_i}}{\sum_{J=1}^{K} e^{z_j}} = a^{(L)}$$

As this function uses the networks values as an exponent of *e*, it provides another layer of nonlinearity for the network. All these values from the layer are added together which is represented by the denominator of the equation. Then each value is inputted into the numerator so that the ratio of that value is compared to the sum of all values in that layer. Since each output value is a ratio of the total value, it causes the output values to be between 0 and 1. This produces the probability of each possible output value. This probability value represents the confidence score that the network has for each output, with a value of 1 the network is 100% confident of the image being that specific Minecraft item (Radečić). This is the layer in which the network uses to make its prediction as it chooses the node that has the highest probability of being that item.

Backpropagation

Backpropagation is the process in which the network begins to learn and increase its accuracy in predictions (3Blue1Brown). This is done by starting at the output and doing the reverse of the functions in the network. Firstly, the output layer and finding the error rates of each output by subtracting the output array by the array of the desired output. An example of this would be since a sword is represented by 3, the array for a correct prediction would be $\begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$ as the

one in the 4th spot representing it as 100% certainty (see Figure 4). The array for the desired output will be represented as *y*. Subtracting the output layer from *y* and squaring it allows for the accuracy of each option to be seen, with a value of zero being completely accurate. This is called the cost function, which I will give the variable *C*.

$$\left(a^{(L)} - y\right)^2 = C$$

With this information, the contribution that the weight and bias had on those values can be found. This is vital for the learning of the network as it can then adjust the weights and bias for the next iteration to achieve a more accurate prediction. The goal is to achieve weights and bias that minimizes the loss values as much as possible.

The algorithm used for finding this minimum is called gradient descent (3Blue1Brown). The minimum is found by going through nonlinear functions, which was done in forward propagation, then doing the reverse by sending the output data through the derivatives of the forward functions. This takes the values given at each layer and finds the slope at that value. This is why the introduction of nonlinear equations to the network is important so that these minimums occur. By adding nonlinear equations to the network, it increases the network's ability to make more complex decisions. This is because input and output values do not have a linear relationship as images don't directly correlate with an index item, so the introduction of nonlinear functions is necessary in a neural network this complex. Since the goal is to find the minimum of this function, finding whether the value is positive or negative is crucial in the learning process. By knowing this information, the network will be able to know which direction that the weights and bias needs to shift to get closer to that slope of 0. This also explains how the network decides how much to shift those values by as the farther away it is from 0, the steeper

the slope, changing the amount that the shift will need to be. The impact the weight had on the cost is found from the rate at which the cost is changing in respect to the weight.

$$\frac{dC}{dw^{(L)}}$$

Since this function consists of layers of ratios in which the weight affects each layer of the network, the first being $z^{(L)}$, then $a^{(L)}$, which finally results in the cost (C). As the network is just a sequence of composite functions strung together, it is possible to reverse the network to find the effect of weights and biases on the cost by using derivatives, using the chain rule in specific. The chain rule is applied since it allows to find the derivative of composite functions and account for all the layers (3Blue1Brown).

$$\frac{dC}{dw^{(L)}} = \frac{dz^{(L)}}{dw^{(L)}}\frac{da^{(L)}}{dz^{(L)}}\frac{dC}{da^{(L)}}$$

This is the chain rule applied to the layers of the network to find the cost's rate of change in respect to the weight.

$$\frac{dz^{(L)}}{dw^{(L)}} = \left(w^{(L)}a^{(L-1)} + b^{(L)}\right)'$$

$$\frac{dz^{(L)}}{dw^{(L)}} = a^{(L-1)}$$

$$\frac{da^{(L)}}{dz^{(L)}} = \left(\begin{cases} 0, & z^{(L)} < 0 \\ z^{(L)}, & z^{(L)} \geq 0 \end{cases}\right)'$$

$$\frac{da^{(L)}}{dz^{(L)}} = \begin{cases} 0, & z^{(L)} < 0 \\ 1, & z^{(L)} > 0 \end{cases} = R'\left(z^{(L)}\right)$$

$$\frac{dC}{da^{(L)}} = \left(\left(a^{(L)} - y\right)^2\right)'$$

$$\frac{dC}{da^{(L)}} = 2\left(a^{(L)} - y\right)\left(a^{(L)} - y\right)'$$

$$\frac{dC}{da^{(L)}} = 2\left(a^{(L)} - y\right)(1)$$

$$\frac{dC}{da^{(L)}} = 2\left(a^{(L)} - y\right)$$

$$\frac{dC}{dw^{(L)}} = a^{(L-1)}R'\left(z^{(L)}\right)2\left(a^{(L)} - y\right)$$

This process is repeated for the bias.

$$\frac{dC}{db^{(L)}} = \frac{dz^{(L)}}{db^{(L)}}\frac{da^{(L)}}{dz^{(L)}}\frac{dC}{da^{(L)}}$$

$$\frac{dz^{(L)}}{db^{(L)}} = \left(w^{(L)}a^{(L-1)} + b^{(L)}\right)'$$

$$\frac{dz^{(L)}}{db^{(L)}} = 1$$

$$\frac{dC}{db^{(L)}} = R'\left(z^{(L)}\right)2\left(a^{(L)} - y\right)$$

The weights and bias are now modified using these values for the next iteration, with *n* being the iteration number. This is the function for gradient descent.

$$w_{n+1}^{(L)} = w_n^{(L)} - \frac{dC}{dw_n^{(L)}}$$

This cycle continues for as many interactions the user sets, allowing for more learning and an increase in accuracy with each iteration.

Experiment

For this paper, I wrote a neural network that tries to determine the item type of 5 different Minecraft items. I got the data by pulling the textures of these items from different Minecraft texture packs that are used to alter the look of the original game.

Now I will go through what the first iteration of an image going through the network might look like. I'll be using the following image of a Minecraft pickaxe to demonstrate this.



Figure 6. Example Minecraft texture of a pickaxe.

This item after it has been converted to black and white is represented as an array. After the arrays are flattened into 1 and divided by 255, this item is shown as the following array.

```
[[0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
  0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
  0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
  0.    0.    0.184 0.184 0.184 0.184 0.184 0.    0.    0.    0.    0.
  0.    0.    0.    0.    0.    0.184 0.69  0.584 0.525 0.525 0.584 0.184
  0.22  0.314 0.    0.    0.    0.    0.    0.    0.    0.    0.184 0.11
  0.11  0.11  0.525 0.525 0.416 0.122 0.    0.    0.    0.    0.    0.
  0.    0.    0.    0.    0.    0.    0.22  0.584 0.525 0.11  0.    0.
  0.    0.    0.    0.    0.    0.    0.    0.    0.    0.22  0.314 0.122
  0.525 0.584 0.11  0.    0.    0.    0.    0.    0.    0.    0.    0.
  0.22  0.416 0.122 0.    0.11  0.525 0.11  0.    0.    0.    0.    0.
  0.    0.    0.    0.22  0.314 0.122 0.    0.    0.11  0.525 0.11  0.
  0.    0.    0.    0.    0.    0.    0.22  0.416 0.122 0.    0.    0.
  0.11  0.584 0.11  0.    0.    0.    0.    0.    0.    0.22  0.314 0.122
  0.    0.    0.    0.    0.11  0.69  0.11  0.    0.    0.    0.    0.
  0.22  0.416 0.122 0.    0.    0.    0.    0.    0.    0.11  0.    0.
  0.    0.    0.    0.22  0.314 0.122 0.    0.    0.    0.    0.    0.
  0.    0.    0.    0.    0.    0.    0.22  0.416 0.122 0.    0.    0.
  0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.122 0.122
  0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
  0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.    0.
  0.    0.    0.    0.   ]]
```

The first step of the network is to generate the weights and bias that will be used. As this is simulating the first iteration, random values are generated for this.

```
def init_params():
    W1 = np.random.rand(5, 256) - 0.5
    b1 = np.random.rand(5, 1) - 0.5
    W2 = np.random.rand(5, 5) - 0.5
    b2 = np.random.rand(5, 1) - 0.5
    return W1, b1, W2, b2
```

Figure 7. Code for creating initial weights and biases.

This is the code that is used to create the weights and bias for the first iteration, with *W* representing weight and *b* representing bias. The function *np.random.rand* generates a matrix of random numbers between 0 and 1, with the parameters representing the dimensions of the matrix. Each of these values then has 0.5 subtracted from them so that the numbers are in the range of -0.5 to 0.5.

```
[[-0.273 -0.267  0.137 ... -0.499 -0.241  0.065]
 [-0.265  0.491 -0.009 ... -0.257 -0.048  0.247]
 [-0.288  0.445 -0.492 ... -0.138 -0.329 -0.384]
 [-0.24   0.108  0.139 ... -0.087 -0.074 -0.317]
 [-0.207 -0.129  0.184 ...  0.351  0.181 -0.398]]

[[-0.308]
 [ 0.194]
 [-0.18 ]
 [-0.141]
 [ 0.494]]

[[-0.303  0.427  0.441  0.479 -0.399]
 [ 0.433 -0.413  0.087  0.203  0.006]
 [-0.294 -0.068  0.137  0.245 -0.036]
 [-0.109 -0.476  0.37   0.105  0.45 ]
 [ 0.429 -0.073  0.139 -0.434  0.332]]

[[-0.236]
 [ 0.272]
 [-0.147]
 [ 0.175]
 [-0.177]]
```

Figure 8. Values for W1, b1, W2, b2.

The first weight and bias (W1 and b1) are then used for the first hidden layer. The equation $w^{(L)}a^{(L-1)} + b^{(L)}$ is applied to all values of the original array. This results in a 5-value array which makes up the 5 nodes in the hidden layer. In this case, the array is $\begin{bmatrix} 0.396 \\ -0.185 \\ 0.806 \\ 0.879 \\ 1.315 \end{bmatrix}$. This array is then inputted into the ReLU activation function which sets all negative numbers equal to 0, as seen in the function $\begin{cases} 0, & z < 0 \\ z, & z \geq 0 \end{cases}$. This results in the final array of the first hidden layer to become $\begin{bmatrix} 0.396 \\ 0 \\ 0.806 \\ 0.879 \\ 1.315 \end{bmatrix}$. This shows that 4 out of 5 of the nodes were activated from this function, though since the first iteration is ultimately random numbers, it doesn't have much meaning in this case.

Next, these values are inputted into the second hidden layer, in which the second weight and bias (W2 and b2) will be applied to the new array using the same function. The array produced from that is $\begin{bmatrix} -0.104 \\ 0.701 \\ 0.015 \\ 1.114 \\ 0.161 \end{bmatrix}$. It is then inputted into the SoftMax activation function $\frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$, resulting in the array $\begin{bmatrix} 0.111 \\ 0.247 \\ 0.125 \\ 0.374 \\ 0.144 \end{bmatrix}$. These values represent the confidence score that the network has

in predicting the item in the image, with each value correlating to the different items (see Figure 4). Since this is the first iteration and the network has no information to work from, the probability values are split evenly among all items, resulting in very inaccurate values. The network chooses the item that has the highest confidence score as its decision, which in this case is the axe with the highest score of 0.374. As seen, the network without any training is very inaccurate and chose the wrong item.

Now that the network has output values, accuracy can be found. Since this is the first iteration and the network has no information to work from, the probability values are split evenly among all items, resulting in very inaccurate values. First, the maximum value of the values in the output layer needs to be found.

```
def get_predictions(A2):
    return np.argmax(A2, 0)
```

Figure 9. Code for finding max value in an array.

The function *np.argmax* returns the index of the maximum value within an array, which in this case returns the type of Minecraft item that the network has the highest confidence score that the image contains, which in this case is the axe with the highest score of 0.374. Therefore, this function returns the index of 1 (see Figure 4). This value is then compared to the index value of the actual item in the image and checks if they are equal. As an image of a pickaxe was inputted for this iteration, the actual index value is 2. The accuracy is then calculated by dividing the number of times that the predicted index numbers were equal to the actual index numbers by the total number of predictions made by the network. As this is only the first iterations and the first prediction was wrong, the network has an accuracy of $\frac{0}{1}$ or 0%.

Now that the image has gone through forward propagation, the cost values need to be found.

$$\left(\begin{bmatrix}0.111\\0.247\\0.125\\0.374\\0.144\end{bmatrix}-\begin{bmatrix}0\\0\\1\\0\\0\end{bmatrix}\right)^2=\left(\begin{bmatrix}0.111\\0.247\\-0.875\\0.374\\0.144\end{bmatrix}\right)^2=\begin{bmatrix}0.0123\\0.0610\\0.766\\0.140\\0.0207\end{bmatrix}$$

These values can be sent through backward propagation to create adjusted weights and biases. First, I will calculate the sensitivity that the weight had on the cost by using the equation that was derived above.

$$\frac{dC}{dw^{(L)}}=a^{(L-1)}R'^{(z^{(L)})}2\left(a^{(L)}-y\right)$$

$$a^{(L-1)}=\begin{bmatrix}0.111\\0.247\\0.125\\0.374\\0.144\end{bmatrix}$$

$$z^{(L)}=\begin{bmatrix}-0.104\\0.701\\0.015\\1.114\\0.161\end{bmatrix}$$

$$R'^{(z^{(L)})}=\begin{cases}0, & z^{(L)}<0\\1, & z^{(L)}>0\end{cases}=\begin{bmatrix}0\\1\\1\\1\\1\end{bmatrix}$$

$$\left(a^{(L)}-y\right)=\begin{bmatrix}0.111\\0.247\\0.125\\0.374\\0.144\end{bmatrix}-\begin{bmatrix}0\\0\\1\\0\\0\end{bmatrix}=\begin{bmatrix}0.111\\0.247\\-0.875\\0.374\\0.144\end{bmatrix}$$

$$\frac{dC}{dw^{(L)}}=\begin{bmatrix}0.396\\0\\0.806\\0.879\\1.315\end{bmatrix}\circ\begin{bmatrix}0\\1\\1\\1\\1\end{bmatrix}\circ 2\begin{bmatrix}0.111\\0.247\\-0.875\\0.374\\0.144\end{bmatrix}=\begin{bmatrix}0\\0\\-1.411\\0.657\\0.379\end{bmatrix}$$

The matrix calculated by subtracting the sensitivity values from the original weight is then used as the new weight for the next iteration.

$$\begin{bmatrix} -0.303 & 0.427 & 0.441 & 0.479 & -0.399 \\ 0.433 & -0.413 & 0.087 & 0.203 & 0.006 \\ -0.294 & -0.068 & 0.137 & 0.245 & -0.036 \\ -0.109 & -0.476 & 0.370 & 0.105 & 0.450 \\ 0.429 & -0.073 & 0.139 & -0.434 & 0.332 \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ -1.411 \\ 0.657 \\ 0.379 \end{bmatrix} = \begin{bmatrix} -0.303 & 0.427 & 0.441 & 0.479 & -0.399 \\ 0.433 & -0.413 & 0.087 & 0.203 & 0.006 \\ 1.117 & 1.343 & 1.548 & 1.656 & 1.375 \\ -0.766 & -1.133 & -0.287 & -0.552 & -0.207 \\ 0.050 & -0.452 & -0.240 & -0.813 & -0.047 \end{bmatrix}$$

This process is repeated for the bias.

$$\frac{dC}{db^{(L)}} = R'\left(z^{(L)}\right)2\left(a^{(L)} - y\right)$$

$$\begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \circ 2 \begin{bmatrix} 0.228 \\ 0.235 \\ -0.809 \\ 0.194 \\ 0.152 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.470 \\ -1.618 \\ 0.388 \\ 0.304 \end{bmatrix}$$

$$\begin{bmatrix} -0.236 \\ 0.272 \\ -0.147 \\ 0.175 \\ -0.177 \end{bmatrix} - \begin{bmatrix} 0 \\ 0.470 \\ -1.618 \\ 0.388 \\ 0.304 \end{bmatrix} = \begin{bmatrix} -0.236 \\ -0.198 \\ 1.471 \\ -0.213 \\ -0.481 \end{bmatrix}$$

The weights and biases used within a network are tweaked editions of the previous iteration's weights and biases, which is how the network can produce better accuracy over time as the network learns what adjustments to make to those values to increase the accuracy. The accuracy of a network depends on the number of iterations it goes through, which is set by the user, as well as the number of images that the network is given to learn from. The more iterations and images the better, as the network has more information to begin learning and produces more accurate results.

For the experiment, I gathered 168 different textures that would be used for the network to learn and test on. This data went through 1000 iterations in which the network was able to learn and predict the items. The accuracy of the network was recorded every 10 iterations.

Results

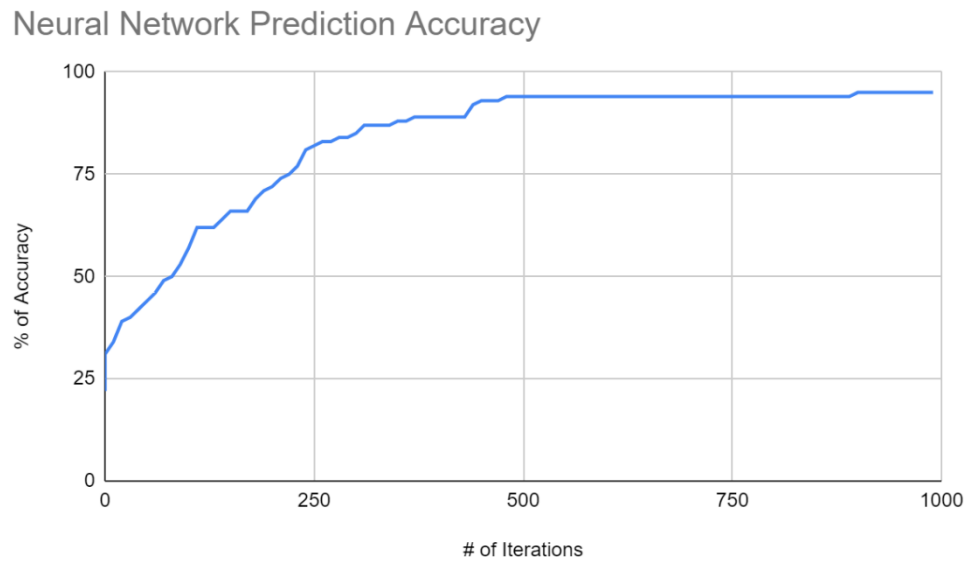


Figure 10. Graph of the accuracy of predictions over the iterations of network.

As seen by the graph, by the end of the 1000 iterations of the network, it was able to predict the item very consistently, with it getting up to 95% accuracy by the end. Within the graph, the rate of change towards 0 is much higher and gradually gets smaller until it is eventually very small. The reasoning for why the network would work like this is because in the early iterations, the cost values are going to be much higher, therefore the learning increases faster since the weights and bias must be adjusted more drastically. This explains why the network experiences a plateau as the number of iterations increases, as the network has learned the bulk of information needed and now only makes small adjustments to increase the accuracy.

Conclusion

Within the realm of machine learning, having 168 different testing data is a very small number to have and often requires many thousands of testing data to create an accurate network. The reason for why I believe that my network was able to work so well despite have a low

amount of data, is due to the type of data that I used. In many instances, the textures between different texture packs don't differ all that much. Most of the time the differences are only within the colors of the item and the shape is unaffected completely, and because of this I believe it makes it much easier for the network to learn the patterns.

```
[[[  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
  [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
  [  0   0   0   0   0   0   0  47  47  47  47  47   0   0   0   0]
  [  0   0   0   0   0   0  47 176 149 134 134 149  47  56  80   0]
  [  0   0   0   0   0   0   0  47  28  28  28 134 134 106  31   0]
  [  0   0   0   0   0   0   0   0   0   0   0  56 149 134  28   0]
  [  0   0   0   0   0   0   0   0   0   0  56  80  31 134 149  28]
  [  0   0   0   0   0   0   0   0   0  56 106  31   0  28 134  28]
  [  0   0   0   0   0   0   0   0  56  80  31   0   0  28 134  28]
  [  0   0   0   0   0   0   0  56 106  31   0   0   0  28 149  28]
  [  0   0   0   0   0   0  56  80  31   0   0   0   0  28 176  28]
  [  0   0   0   0   0  56 106  31   0   0   0   0   0   0  28   0]
  [  0   0   0  56  80  31   0   0   0   0   0   0   0   0   0   0]
  [  0   0  56 106  31   0   0   0   0   0   0   0   0   0   0   0]
  [  0   0  31  31   0   0   0   0   0   0   0   0   0   0   0   0]
  [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]]]
```

```
[[[  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
  [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
  [  0   0   0   0   0   0   0  96  96  96  96  96   0   0   0   0]
  [  0   0   0   0   0   0  96 239 220 176 176 220  96  56  80   0]
  [  0   0   0   0   0   0   0  96  47  47  47 176 176 106  31   0]
  [  0   0   0   0   0   0   0   0   0   0   0  56 220 176  47   0]
  [  0   0   0   0   0   0   0   0   0   0   0  56  80  31 176 220]
  [  0   0   0   0   0   0   0   0   0   0  56 106  31   0  47 176]
  [  0   0   0   0   0   0   0   0   0  56  80  31   0   0  47 176]
  [  0   0   0   0   0   0   0   0  56 106  31   0   0   0  47 220]
  [  0   0   0   0   0   0   0  56  80  31   0   0   0   0  47 239]
  [  0   0   0   0   0  56 106  31   0   0   0   0   0   0  47   0]
  [  0   0   0  56  80  31   0   0   0   0   0   0   0   0   0   0]
  [  0   0  56 106  31   0   0   0   0   0   0   0   0   0   0   0]
  [  0   0  31  31   0   0   0   0   0   0   0   0   0   0   0   0]
  [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]]]
```

Figure 11. Comparison of the interpretation of how items look when only the color is changed.

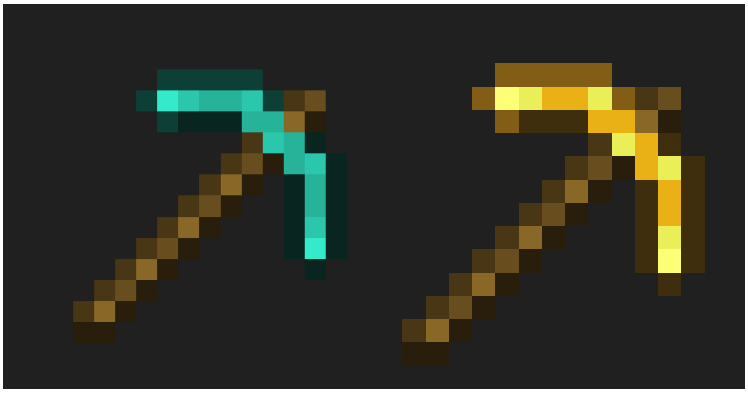


Figure 12. The actual images being compared in Figure 11.

```
[[[  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
  [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
  [  0   0   0   0   0   0   0  47  47  47  47  47   0   0   0   0]
  [  0   0   0   0   0   0  47 176 149 134 134 149  47  56  80   0]
  [  0   0   0   0   0   0   0  47  28  28  28 134 134 106  31   0]
  [  0   0   0   0   0   0   0   0   0   0   0  56 149 134  28   0]
  [  0   0   0   0   0   0   0   0   0   0  56  80  31 134 149  28]
  [  0   0   0   0   0   0   0   0   0  56 106  31   0  28 134  28]
  [  0   0   0   0   0   0   0   0  56  80  31   0   0  28 134  28]
  [  0   0   0   0   0   0   0  56 106  31   0   0   0  28 149  28]
  [  0   0   0   0   0   0  56  80  31   0   0   0   0  28 176  28]
  [  0   0   0   0  56 106  31   0   0   0   0   0   0   0  28   0]
  [  0   0   0  56  80  31   0   0   0   0   0   0   0   0   0   0]
  [  0   0  56 106  31   0   0   0   0   0   0   0   0   0   0   0]
  [  0   0  31  31   0   0   0   0   0   0   0   0   0   0   0   0]
  [  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]]]
```

```
[[[  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
  [  0   0   0   0 212 234 234 212 234  44  61  44   0  44  73   0]
  [  0   0   0   0   0   0 155 185 185 185 135  73  73  61  44   0]
  [  0   0   0   0   0   0   0   0   0   0   0 212 135 112  73 185]
  [  0   0   0   0   0   0   0   0   0   0   0  44  61  44  44  44]
  [  0   0   0   0   0   0   0   0   0   0   0  61  73  44 135 112]
  [  0   0   0   0   0   0   0   0   0  68  73  44   0 185 135  44]
  [  0   0   0   0   0   0   0   0   0  77  95  55   0   0 212 185]
  [  0   0   0   0   0   0   0   0   0  87  95  68   0   0   0   0]
  [  0   0   0   0   0   0  77  95  68   0   0   0   0   0 155 212]
  [  0   0   0   0   0  77  95  68   0   0   0   0   0   0   0 234]
  [  0   0   0   0  68  95  55   0   0   0   0   0   0   0   0 185]
  [  0   0  68  95  55   0   0   0   0   0   0   0   0   0   0   0]
  [  0  68  87  55   0   0   0   0   0   0   0   0   0   0   0   0]
  [ 61  87  55   0   0   0   0   0   0   0   0   0   0   0   0   0]
  [ 73  44   0   0   0   0   0   0   0   0   0   0   0   0   0   0]]]
```

Figure 13. Comparison of the interpretation of how items look when the color and shape are changed.

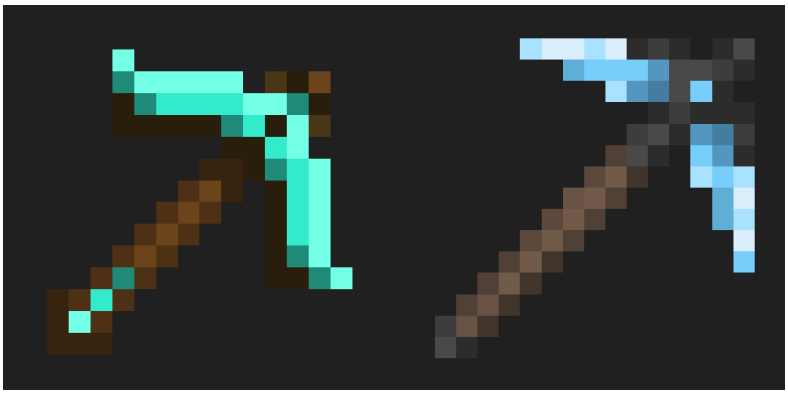


Figure 14. The actual images being compared in Figure 13.

As seen, whenever the texture involves changing the shape of the item, it has a much more drastic effect on the data than only having a change in color. This is showcased by the fact that with unique shapes, values that would instead be 0, would have high value and vice versa. When the texture only involves changing the colors of the item, the fluctuation of the values is much slighter as it doesn't involve changing to or from the value 0.

Due to this, the accuracy that the network can produce depends heavily on the type of textures that I give it to test. If I were to include more data that involves textures that have major differences in shape, though the network has more data to learn from, it would also produce lower accuracy for the earlier iterations as it the data would differ greatly from the previous data.

Though it would likely produce lower accuracy, it would better generalize across all texture packs as it would contain those unique cases where the textures drastically differ, therefore would be able to more accurately predict textures with more unique shapes. In regard to the images that the network processed, it showed great accuracy and would confidently be able to decipher the item types of the majority of images that it is given.

There are many improvements that could be made to make the network more accurate and inclusive of different texture types. One of which would be using much more testing data as well as a much more diverse data set to improve the networks generalization abilities in its predictions. This would be as simple as importing more images into the data set and rerunning the network through all the iterations. Another improvement that could be made to the network is increasing the complexity by either implementing more hidden layers with nonlinear functions, creating a deeper learning aspect to each iteration for the network.

Overall, after going through enough iterations, the neural network can produce very accurate predictions on the item type within the image. This shows how powerful the use of machine learning can be as it can pick up patterns from images and differ it from other images. An example where this was an important tool would be in the medical field. Machine learning can be used to train a neural network about x-rays and different conditions that can be found by them, and with enough data would be able to accurately diagnose the patient (GAO). Though this would bring up conversation about ethical issues on whether these results should be trusted, it would provide a good starting point for finding the possibilities of said condition. Machine learning is and will continue to become a large part of our lives and the technology we use, therefore knowing how they work is very beneficial.

Appendix A

The code for the neural network.



```python
import numpy as np
import pandas as pd
import os
import math
from PIL import Image
from matplotlib import pyplot as plt

#uses kaggle data base to insert all images into the network

data = []
for i in os.listdir('/kaggle/input/trainingdata'):
    imgname = i
    img = Image.open('/kaggle/input/trainingdata/'+imgname).convert('L')
    itemtype = None
    if 'shovel' in imgname:
        itemtype = 0
    if 'axe' in imgname:
        itemtype = 1
    if 'pickaxe' in imgname:
        itemtype = 2
    if 'sword' in imgname:
        itemtype = 3
    if 'hoe' in imgname:
        itemtype = 4
    imgarr = np.asarray(img).flatten()
    imgarr = np.insert(imgarr, 0, itemtype)
    data.append(imgarr)
data = np.array(data)

m, n = data.shape
#m is equal to the number of images in dataset, n is the # of pixels in image
np.random.shuffle(data)

datadev = data[0:1].T
Xdev = datadev[1:n]
Ydev = datadev[0]
Xdev = Xdev / 255

datatrain = data[1:m].T
Ytrain = datatrain[0]
Xtrain = datatrain[1:n]
Xtrain = Xtrain / 255

def init_params():
    W1 = np.random.rand(5, 256) - 0.5
    b1 = np.random.rand(5, 1) - 0.5
    W2 = np.random.rand(5, 5) - 0.5
    b2 = np.random.rand(5, 1) - 0.5
    return W1, b1, W2, b2

def ReLU(Z):
    return np.maximum(Z, 0)

def softmax(Z):
    A = np.exp(Z) / sum(np.exp(Z))
    return A

def forward_prop(W1, b1, W2, b2, X):
    Z1 = W1.dot(X) + b1
    A1 = ReLU(Z1)
    Z2 = W2.dot(A1) + b2
    A2 = softmax(Z2)
    return Z1, A1, Z2, A2

def ReLU_deriv(Z):
    return Z > 0

def one_hot(Y):
    one_hot_Y = np.zeros((Y.size, Y.max() + 1))
    one_hot_Y[np.arange(Y.size), Y] = 1
    one_hot_Y = one_hot_Y.T
    return one_hot_Y

def backward_prop(Z1, A1, Z2, A2, W1, W2, X, Y):
    one_hot_Y = one_hot(Y)
    dZ2 = A2 - one_hot_Y
    dW2 = 1 / m * dZ2.dot(A1.T)
    db2 = 1 / m * np.sum(dZ2)
    dZ1 = W2.T.dot(dZ2) * ReLU_deriv(Z1)
    dW1 = 1 / m * dZ1.dot(X.T)
    db1 = 1 / m * np.sum(dZ1)
    return dW1, db1, dW2, db2

def update_params(W1, b1, W2, b2, dW1, db1, dW2, db2, alpha):
    W1 = W1 - alpha * dW1
    b1 = b1 - alpha * db1
    W2 = W2 - alpha * dW2
    b2 = b2 - alpha * db2
    return W1, b1, W2, b2

def get_predictions(A2):
    return np.argmax(A2, 0)

def get_accuracy(predictions, Y):
    print(predictions, Y)
    return np.sum(predictions == Y) / Y.size

def gradient_descent(X, Y, alpha, iterations):
    W1, b1, W2, b2 = init_params()
    for i in range(iterations):
        Z1, A1, Z2, A2 = forward_prop(W1, b1, W2, b2, X)
        dW1, db1, dW2, db2 = backward_prop(Z1, A1, Z2, A2, W1, W2, X, Y)
        W1, b1, W2, b2 = update_params(W1, b1, W2, b2, dW1, db1, dW2, db2, alpha)
        if i % 10 == 0:
            print("Iteration: ", i)
            predictions = get_predictions(A2)
            print(get_accuracy(predictions, Y))
    return W1, b1, W2, b2

W1, b1, W2, b2 = gradient_descent(Xtrain, Ytrain, 0.1, 2000)
```

Works Cited

3Blue1Brown. “Backpropagation Calculus: Chapter 4, Deep Learning.” *YouTube*, 3 Nov. 2017, https://youtu.be/tIeHLnjs5U8.

3Blue1Brown. “What Is Backpropagation Really Doing?: Chapter 3, Deep Learning.” *YouTube*, 3 Nov. 2017, https://youtu.be/Ilg3gGewQ5U.

D2L. “5.3. Forward Propagation, Backward Propagation, and Computational Graphs¶ Colab [Pytorch] Open the Notebook in Colab Colab [Mxnet] Open the Notebook in Colab Colab [Jax] Open the Notebook in Colab Colab [Tensorflow] Open the Notebook in Colab Sagemaker Studio Lab Open the Notebook in SageMaker Studio Lab.” *5.3. Forward Propagation, Backward Propagation, and Computational Graphs - Dive into Deep Learning 1.0.0-beta0 Documentation*, https://d2l.ai/chapter_multilayer-perceptrons/backprop.html.

Divakar, Varun, and Rekhit Pachanekar. “Forward Propagation in Neural Networks.” *Quantitative Finance & Algo Trading Blog by QuantInsti*, 17 Mar. 2022, https://blog.quantinsti.com/forward-propagation-neural-networks/.

GAO. “Machine Learning's Potential to Improve Medical Diagnosis.” *U.S. GAO*, 8 Dec. 2022, https://www.gao.gov/blog/machine-learnings-potential-improve-medical-diagnosis.

Gavrilova, Yulia. “What Are Convolutional Neural Networks?” *Serokell Software Development Company*, Serokell, 3 Aug. 2021, https://serokell.io/blog/introduction-to-convolutional-neural-networks.

Gupta, Dishashree. “Fundamentals of Deep Learning - Activation Functions and When to Use Them?” *Analytics Vidhya*, 1 Dec. 2022, https://www.analyticsvidhya.com/blog/2020/01/fundamentals-deep-learning-activation-functions-when-to-use-them/.

IBM. “What Are Neural Networks?” *IBM*, https://www.ibm.com/topics/neural-networks.

Math Insight. “The Transpose of a Matrix.” *Math Insight*, https://mathinsight.org/matrix_transpose.

Radečić, Dario. “Softmax Activation Function Explained.” *Towards Data Science*, 21 Mar. 2022, https://towardsdatascience.com/softmax-activation-function-explained-a7e1bc3ad60.

Tiwari, Sakshi. “Activation Functions in Neural Networks.” *GeeksforGeeks*, GeeksforGeeks, 17 Feb. 2023, https://www.geeksforgeeks.org/activation-functions-neural-networks/.

Zhang, Samson. “Building a Neural Network from Scratch (No Tensorflow/Pytorch, Just Numpy & Math).” *YouTube*, 24 Nov. 2020, https://youtu.be/w8yWXqWQYmU.